MobileExplorer: Accelerating On-Device Inference for Mobile GUI Agents via Online Exploration

ABSTRACT

Mobile graphical user interface (GUI) agents enable AI models to autonomously operate smartphones on behalf of users. However, most existing systems focus primarily on task accuracy and rely on remote servers for inference, introducing privacy risks and network-dependent latency. As a result, fully on-device mobile GUI agents remain underexplored. We propose MobileExplorer, a fully on-device framework that improves mobile GUI task automation via latency-aware online exploration. The key idea is to exploit the long per-step reasoning time of vision-language models (VLMs) by performing lightweight, time-budgeted exploration in parallel with model inference. During exploration, the agent probes semantically relevant UI elements within the reasoning window and records state transitions as structured exploration memory. To preserve execution consistency in live mobile environments, we design a two-level rollback mechanism that reliably restores the starting UI state when an explored branch does not align with the final VLM decision. We evaluate MobileExplorer on multiple real-world devices using the AndroidWorld benchmark. Under fully on-device execution, our system preserves task success while reducing the average number of interaction steps by approximately 23%.

1 INTRODUCTION

Mobile GUI agents have recently emerged with the rapid progress of large language models (LLMs) [1, 6] and vision-language models (VLMs) [2, 17], enabling an end-to-end paradigm for mobile task automation where task understanding, planning, and execution are unified within a single model [21]. Existing agents typically adopt two input modalities: LLM-based methods rely on accessibility trees [4, 5, 9, 21, 22], while VLM-based methods directly operate on screenshots [19, 20, 26, 27, 32]. However, most current mobile GUI agents execute LLMs or VLMs on remote servers, requiring on-device data to be uploaded through the network, which introduces privacy and security risks. Therefore, a natural direction is to deploy mobile GUI agents fully on-device, so that perception, reasoning, and action can be performed locally without external data transmission.

However, building an on-device mobile GUI agent is still challenging. First, many on-device agents rely mainly on LLMs with textual inputs such as accessibility trees, which introduce an inherent information bottleneck by abstracting away rich visual layouts, spatial structures, icons, and other non-textual interface cues. Such text-centric representations are also often incomplete, device-dependent, and unable to faithfully reflect visually styled or dynamically rendered elements, limiting robustness in real-world mobile environments. This motivates our focus on screenshot-based perception and VLMs. Prior work on mobile UI understanding shows that raw screenshots contain rich visual context, including layout organization, spatial relations, icons, and non-textual cues that are essential for interpreting complex interfaces [3, 10, 28]. In contrast, accessibility trees mainly expose textual attributes and structural metadata, whereas screenshots provide a holistic and faithful representation of the actual interface. Moreover, screenshot inputs can be adaptively scaled (e.g., resolution or region selection) to balance accuracy and efficiency under on-device resource constraints. Empirical studies in vision-language UI understanding further demonstrate that screenshot-based models achieve stronger visual grounding and often outperform hierarchy-based approaches on complex GUI tasks [8, 10]. These findings suggest that screenshot-based VLMs enable more human-like visual interpretation and improve task success in visually structured scenarios.

Second, although VLMs provide stronger visual understanding, they introduce significantly higher computational and memory costs than text-only models, making continuous on-device execution still expensive even for recent compact models [26, 32]. To alleviate this cost, recent GUI agents have explored several acceleration strategies, such as reducing the number of model calls through multi-step planning or script-style execution [21, 22], transforming action generation into lightweight verification over candidate actions [4], and pruning redundant visual or contextual tokens to lower inference complexity in vision-language pipelines [13]. However, these approaches still have inherent limitations. Multi-step planning or script-style execution can

be brittle under dynamic and unseen UI states, verifier-based pipelines remain dependent on candidate action quality and still incur non-trivial per-step reasoning overhead, and aggressive token or context pruning may discard fine-grained visual cues that are critical for precise GUI grounding. Third, existing on-device agents usually block and wait during model reasoning, leaving sensing and interaction resources idle and underutilized. Finally, most existing systems follow a strictly sequential perception–reasoning–action loop, which further limits system-level efficiency under tight latency and resource constraints on mobile devices.

In this paper, we propose MobileExplorer, a novel on-device mobile GUI agent framework that utilizes the long reasoning latency of the model to perform online exploration in parallel. Instead of remaining idle during model inference, the system proactively explores the current screen to collect task-relevant information. To achieve effective exploration under strict latency budgets, MobileExplorer adopts a time-bounded and diversity-aware exploration strategy that prioritizes semantically relevant and interactive UI elements based on lightweight text embeddings [16]. During exploration, the system maintains stable navigation through controlled rollback and state recovery to avoid drifting from the original task context, addressing the system challenges caused by speculative interactions. The discovered UI elements and interaction traces are then stored in a structured form and summarized into compact textual hints, which can be seamlessly appended to the prompt to enhance subsequent reasoning. By explicitly externalizing exploration knowledge and reusing it as contextual signals, MobileExplorer improves decision accuracy without introducing additional model inference overhead.

We deploy our system on a commercial off-the-shelf smartphone for real-world evaluation. We further evaluate on the widely used emulator-based benchmark Android World [14], where our method achieves accuracy under a latency of latency.

The main contributions of this work are:

- We present MobileExplorer, an on-device mobile GUI agent framework that utilizes model reasoning latency to perform parallel online exploration, improving system efficiency without increasing model invocation frequency.
- We design an online exploration mechanism that prioritizes semantically relevant and diverse UI elements within limited inference time, maintains stable task context through controlled rollback, and converts exploration results into structured hints for subsequent reasoning.
- We implement a full end-to-end system that overlaps perception, reasoning, and exploration on resource-constrained mobile devices, reducing idle time in the conventional sequential execution pipeline.
- We conduct extensive evaluation across multiple commercial devices and the Android World benchmark, demonstrating improved task success, reduced interaction latency, and fewer execution steps under realistic mobile constraints.

2 RELATED WORK

Mobile GUI Agents. Recent advances in foundation models have made mobile GUI agents an emerging research topic [4, 5, 8, 9, 19, 21, 22, 26, 27, 32]. Early works mainly rely on LLMs with structured inputs such as accessibility trees and action histories. For example, AutoDroid [21] leverages memory and action history to support long-horizon GUI interaction, while AutoDroid-V2 [22] reduces latency by replacing iterative multi-step reasoning with more direct planning. V-Droid [4] further adopts a verification-based paradigm, where the model selects actions from candidate UI elements instead of directly generating operations. More recent studies explore VLM-based agents that operate directly on screenshots, enabling visual grounding over layout, icons, and spatial structures without relying on UI trees. CogAgent [8] aligns user instructions with screen regions through vision-language attention, and Mobile-Agent-V3 [27] employs a collaborative architecture to handle perception and reasoning in complex tasks. In addition, MAI-UI [32], TARS-UI [19], and STEP-UI [26] train VLM-based agents with interaction trajectories and reinforcement signals, improving robustness in real-world GUI environments.

On-device deployment of mobile GUI agent. Most existing mobile GUI agent systems follow a server–device architecture, where the agent logic runs on a remote server and interacts with the mobile phone through Android Debug Bridge (ADB) [7]. While this design simplifies computation, it requires uploading device data to the server, raising privacy concerns and limiting deployment in sensitive or disconnected scenarios. A few recent studies have examined the reasoning latency of language models directly on mobile devices [21, 22]. However, these works focus only on LLM inference and do not consider vision-language models (VLMs), which are essential for perception in GUI tasks. Moreover, they do not provide a complete end-to-end on-device framework

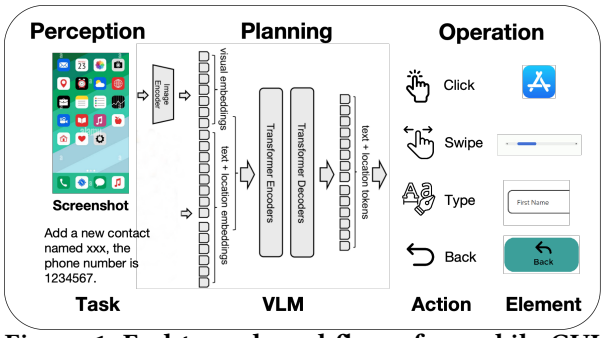


Figure 1: End-to-end workflow of a mobile GUI agent. The agent takes a screenshot as visual input, performs multimodal reasoning, and outputs text that are mapped to executable GUI actions.

that integrates perception, reasoning, and action execution within the mobile device itself.

Offline exploration for mobile GUI agents. Exploration has been studied as a way to augment GUI agents with additional interaction knowledge beyond pretrained model priors. GUI-explorer [23] first performs autonomous and systematic exploration over the application to collect function-aware trajectories, and then mines transition-aware knowledge from the collected interaction traces through unsupervised analysis of state–action–outcome triples. The extracted knowledge is organized into a knowledge store and later retrieved to guide task execution, rather than being generated during real-time interaction. Similarly, LLM-Explorer [31] conducts large-scale app exploration to build an abstract knowledge base, including UI states, actions, and interaction graphs, which is maintained and updated from historical exploration traces and used to guide future action selection. Other knowledge-augmented agents [30] also rely on pre-collected trajectories or retrieved experience as static external knowledge for decision making. Technically, these methods decouple exploration from online task execution: exploration is conducted beforehand to construct reusable knowledge repositories, and the agent mainly performs retrieval or planning at runtime. In contrast, we design an online exploration mechanism that runs concurrently with model inference, enabling the agent to actively probe the current UI state and acquire task-relevant information during execution instead of relying on pre-built offline knowledge.

3 A MOTIVATION STUDY

3.1 Background

3.1.1 Workflow of Mobile GUI Agent. Figure 1 illustrates the typical end-to-end workflow of a mobile GUI agent. Given the current screen of a mobile phone, the agent first captures a screenshot that reflects the visual layout, icons, text, and spatial relationships among UI elements. An image encoder transforms the screenshot into visual embeddings, which encode appearance and structural information of the interface.

These visual embeddings are combined with textual tokens, such as user instructions, task descriptions, or dialogue context. Positional encodings are further incorporated to preserve spatial relationships among interface elements. The resulting multimodal token sequence is processed by transformer-based encoders and decoders to perform joint visual–textual reasoning over the GUI state.

Based on this reasoning process, the model generates output tokens that correspond to interaction decisions. These tokens typically include both semantic components (e.g., the intended action type) and spatial components (e.g., the location of the target element). They are then translated into executable GUI operations, such as tapping an icon, entering text, adjusting a control, or navigating between pages. This perception–reasoning–action loop forms the core workflow of modern mobile GUI agents.

3.1.2 Benchmark for mobile GUI agents. Recent benchmarks such as Android-in-the-Wild [15] and AndroidControl [12] have enabled systematic evaluation of mobile GUI agents across diverse real-world applications. However, these benchmarks rely on offline interaction traces and therefore do not reflect the dynamic execution conditions of real mobile devices, such as runtime delays, system states, and environment-dependent behaviors.

To better capture these dynamics, interactive platforms including Android World [14] and Android Lab [25] provide environments where agents directly operate real Android applications through system-level control interfaces. Such platforms allow evaluation under realistic execution conditions, including live UI transitions and system responses.

In this work, we adopt Android World as the primary evaluation platform to assess our system under dynamic, real-device interaction settings.

3.2 Why Vision is Essential for Mobile GUI Agents

Mobile GUI interaction is fundamentally a visually grounded problem. As shown in Fig. 2, the top-performing agents on the AndroidWorld benchmark are predominantly screenshot-based systems. In contrast, approaches that

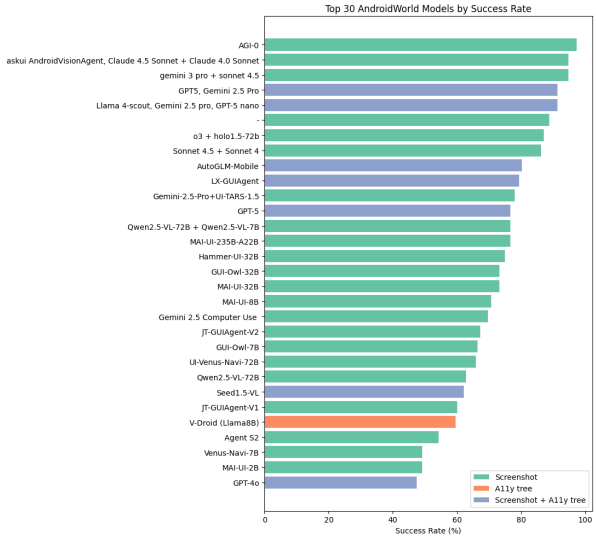
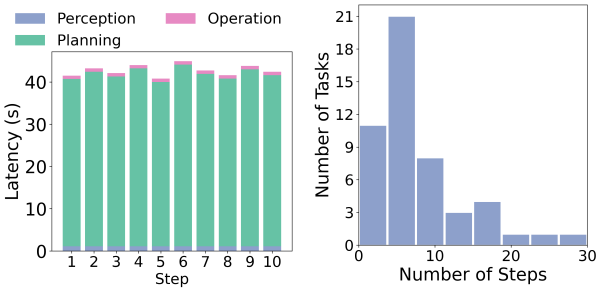


Figure 2: Top models on AndroidWorld [4, 11, 18, 19, 24, 27, 32]. Leading agents are predominantly VLM-based, highlighting the importance of visual understanding over tree-only representations in mobile GUI tasks.

rely mainly on accessibility trees or textual UI representations rarely appear among the leading entries. This trend suggests that high task success in realistic mobile environments is largely determined by the agent's ability to understand rendered visual content rather than structured UI trees alone.

This dominance of vision stems from the nature of GUI tasks. Many interactions require recognizing visual elements whose semantics are only partially reflected in accessibility metadata, such as icons, styled text, or transient visual prompts. Furthermore, mobile tasks often depend on spatial and layout cues (e.g., relative positions, corner locations, grouping of elements), which are naturally encoded in pixel space but only weakly captured in tree structures. In addition, successful interaction frequently requires interpreting visual feedback after each action, such as state changes or confirmation indicators on the updated screen. These characteristics make mobile GUI automation inherently vision-centric, explaining why VLM-based agents consistently outperform LLM-only agents in complex task settings.

Taken together, these observations indicate that vision is not optional but a fundamental capability for reliable mobile GUI agents.



(a) Latency of task "Record an audio and save it". **(b) Steps statistics in AndroidWorld [14].**

Figure 3: The repeated long reasoning phases create idle intervals that can be exploited for parallel exploration.

3.3 Opportunity for Online Exploration

While visual reasoning enables accurate GUI understanding, it introduces a critical systems bottleneck on mobile devices: large and repeated on-device reasoning latency. As shown in Fig. 3a, even for a simple mobile task such as "Record an audio and save it", each decision step incurs substantial planning time, which dominates the end-to-end latency (around tens of seconds per step), while perception and operation only take a small fraction of the total time. This indicates that the agent spends most of its time waiting for the VLM to finish reasoning before issuing the next action.

More importantly, mobile GUI tasks are inherently multi-step and sequential. Fig. 3b shows that completing a task often requires many interaction steps, and some tasks exceed 15–20 steps. As a result, the large per-step reasoning latency is repeatedly accumulated across the whole trajectory, leading to extremely high task completion time on resource-constrained devices. Therefore, the inefficiency of on-device GUI agents stems not from a single slow inference, but from the combination of long-horizon decision making and heavy per-step reasoning cost.

We further conduct a preliminary measurement on a commercial smartphone (Samsung Galaxy S24, 12GB RAM) running a 2B VLM (MAI-UI) with on-device inference. As observed in Fig. 3a, a single reasoning step takes around 40 seconds on average, whereas a lightweight UI interaction, such as a tap and the resulting screen transition, only takes about 1–2 seconds. This large gap implies that, within one reasoning step, the system has sufficient time budget to perform roughly 20 lightweight interactions. In other words, a considerable amount of executable time is available but remains unused while the model performs visual reasoning.

This observation reveals a fundamental inefficiency in conventional step-wise GUI agents. During each reasoning phase, the UI environment remains unchanged and the interaction pipeline is largely idle, yet the system passively waits for the model output. The device is effectively blocked by sequential reasoning, even though the UI can already be probed and interacted with during this interval.

We argue that this repeated idle interval should be exploited rather than wasted. Instead of treating reasoning latency as unavoidable overhead, we leverage it as an opportunity to perform lightweight online exploration in parallel with model inference. Concretely, while the VLM is generating the next action, the exploration controller proactively interacts with selected UI elements and observes the resulting screen transitions, gradually uncovering hidden UI branches and structural context. The collected information reduces uncertainty in subsequent decisions, which can shorten the interaction trajectory and reduce unnecessary trial-and-error steps.

However, enabling online exploration during reasoning also introduces several non-trivial system challenges. First, how to perform effective exploration under a strict time budget, so that the limited idle interval is used to gather diverse and task-relevant information. Second, how to convert the discovered UI states and interaction traces into compact knowledge that can be directly leveraged to enhance subsequent reasoning. Third, how to reliably return the agent to the UI state where exploration started, since parallel exploration may trigger screen transitions and change the original interface context. Without accurately restoring the starting UI state, the subsequent reasoning step may operate on a different screen than the one used for inference, leading to inconsistent decisions.

4 SYSTEM OVERVIEW

We propose MobileExplorer, a fully on-device VLM-based mobile GUI agent that overlaps model reasoning with online exploration, as illustrated in Fig. 4. Unlike conventional GUI agents that follow a strictly sequential perception–reasoning–action loop, MobileExplorer treats the long VLM reasoning phase as an exploitable time window and executes exploration concurrently on the same device. All sensing, reasoning, exploration, and interaction are performed locally, preserving user privacy and avoiding any data transmission.

At each step i, the agent receives the current screenshot and task description as input and feeds them to the VLM for action reasoning. Meanwhile, instead of waiting idly for the model output, an online exploration controller is triggered in parallel. The controller parses the current UI elements and computes semantic similarity between candidate interactive elements and the task description using a lightweight text embedding model. Based on this relevance score, the system selects a small set of task-related UI elements and performs lightweight exploratory interactions (e.g., tap and observe screen transitions) to probe hidden UI branches and interface structure.

To ensure consistency with the main decision trajectory, the system explicitly records the exploration traces and screen states, and reliably returns the device to the UI state where exploration started before the next action is executed. This design prevents exploration-induced screen changes from interfering with the original reasoning context, which is critical for multi-step GUI tasks.

The outcomes of exploration, including visited screens, discovered UI semantics, and interaction feedback, are compactly summarized into structured hints (e.g., highlighted elements and textual cues). These hints are appended to the prompt of the next reasoning step, enabling the VLM to leverage newly acquired environmental evidence rather than relying solely on a single static screenshot. In this way, reasoning becomes progressively informed by runtime observations of the UI.

From a system perspective, MobileExplorer forms a partially parallel pipeline: VLM reasoning operates on step i while exploration continuously collects contextual knowledge for future steps. This pipeline-aware design converts otherwise idle reasoning time into productive interaction time, improves decision accuracy through knowledge augmentation, and reduces unnecessary trial-and-error steps in long-horizon mobile GUI tasks, while remaining fully deployable on commodity smartphones.

5 DESIGN OF MOBILEEXPLORER

5.1 Problem Formulation

We consider an on-device mobile GUI agent that interacts with an application through step-wise operations. At step i, the agent observes the current UI screenshot s_i and receives a task description $\mathcal{T}$. Due to the partial observability of mobile interfaces, the full UI structure is unknown a priori and can only be incrementally revealed through interaction.

State and Action Space. At step i, the agent resides in an implicit UI state v_i represented by the screenshot s_i and the set of parsed interactive UI elements:

$$\mathcal{A}_i = \{a_i^{(1)}, a_i^{(2)}, \ldots, a_i^{(K_i)}\}.$$

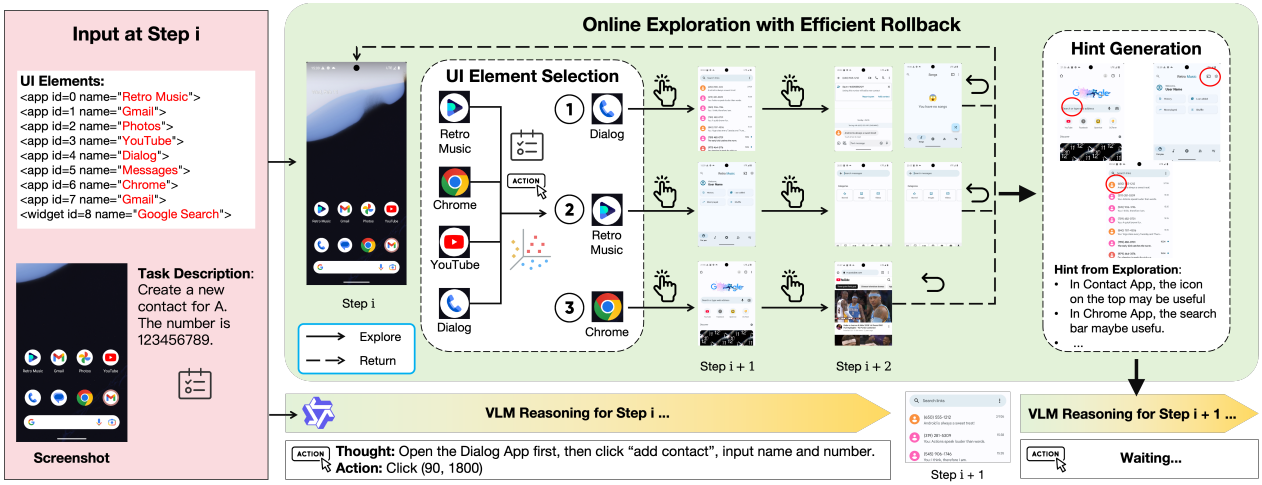


Figure 4: System overview of MobileExplorer. During each reasoning step, the system performs lightweight online exploration in parallel with VLM inference, summarizes discovered screen knowledge into compact hints, and feeds them to the next reasoning step.

Each action corresponds to interacting with a UI element (e.g., click, scroll), which may trigger a screen transition to a new UI state.

Unlike conventional agents that only rely on the current screen, MobileExplorer performs *online local exploration* during model reasoning. A lightweight controller probes selected UI elements and observes resulting screen transitions, forming a local interaction subgraph:

$$\mathcal{G}_i^{\text{local}} = (\mathcal{V}_i^{\text{visited}}, \mathcal{E}_i^{\text{observed}}),$$

where $\mathcal{V}_i^{\text{visited}}$ denotes visited UI states and $\mathcal{E}_i^{\text{observed}}$ denotes observed transitions during exploration.

Latency-Constrained Exploration. On-device VLM inference incurs significant latency. Let τ_i^{vlm} denote the reasoning latency at step i. Online exploration is executed within this idle window:

$$\tau_i^{\text{explore}} \leq \tau_i^{\text{vlm}},$$

so that exploration does not introduce additional user-perceived delay.

Exploration Context Augmentation. Instead of directly expanding the action space, exploration produces compact contextual knowledge summarizing visited UI elements and discovered screen semantics:

$$C_i = \{u_j \mid u_j \in \mathcal{U}_i^{\text{visited}}\},$$

where $\mathcal{U}_i^{\text{visited}}$ denotes task-relevant UI elements observed during exploration up to step i.

Reasoning with Exploration Context. The VLM generates a task-conditioned representation:

$$\mathbf{r}_i = f_{\text{vlm}}(s_i, \mathcal{T}, C_i),$$

where C_i provides exploration-derived hints beyond the current visible screen.

System Objective. The objective of MobileExplorer is to maximize task completion probability while minimizing interaction cost under on-device latency constraints:

$$\max_{\pi} \Pr(v_N \in \mathcal{V}_{\text{goal}}(\mathcal{T})) \quad \text{s.t.} \quad \sum_i \tau_i^{\text{interact}} \leq B,$$

where B is the latency budget. The key idea is to utilize otherwise idle reasoning latency to gather additional UI context online without increasing execution delay.

5.2 Latency-Constrained Goal-Conditioned Exploration

Online exploration operates strictly within the latency budget defined by the VLM reasoning time at each decision step. Since reasoning itself dominates step latency, exploration must utilize only the idle window without introducing additional delay. Exhaustive traversal of the UI graph is infeasible on resource-constrained mobile devices. Therefore, we design a time-budgeted, relevance-driven exploration mechanism that maximizes task-relevant coverage within bounded time, as shown in Figure 5.

At the beginning of each reasoning step, the exploration controller parses the current accessibility tree and extracts all clickable UI elements. Each element is

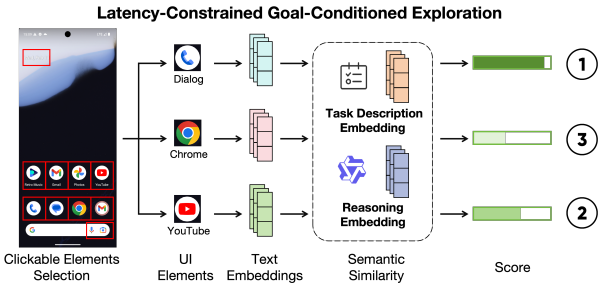


Figure 5: During each VLM reasoning step, the system ranks clickable UI elements by task relevance and performs short exploratory probes under a strict time budget.

converted into a lightweight textual representation (e.g., text label, content description, resource identifier). We compute a semantic similarity score between each element representation and the task description embedding using a lightweight embedding model. This produces a relevance-ranked candidate list.

Instead of uniformly probing all clickable elements, the system prioritizes high-relevance candidates while explicitly maintaining diversity. To prevent redundant traversal, we maintain interaction history at both element level and UI-branch level. Elements that have been previously visited, or that belong to branches already explored in earlier steps, are down-weighted. This history-aware scoring encourages discovery of unseen UI paths while preserving task relevance.

Exploration is executed through short interaction cycles. Each probe consists of a click action followed by observing the resulting screen transition. The updated screen is parsed again to collect additional contextual information. Because each probe is short and bounded, multiple probes can be executed within a single reasoning window. Exploration is terminated immediately once the time budget is exhausted or if an unexpected state transition is detected.

This design enables the agent to uncover deeper or hidden UI branches, collect auxiliary contextual evidence, and build structured exploration memory without increasing end-to-end latency. By coupling semantic prioritization with diversity-aware history control under a strict time budget, the system balances efficiency and coverage in mobile-scale UI environments.

5.3 Exploration with Efficient Rollback

Online exploration in mobile GUI environments fundamentally differs from offline exploration in simulators. In offline settings, agents can rely on environment cloning

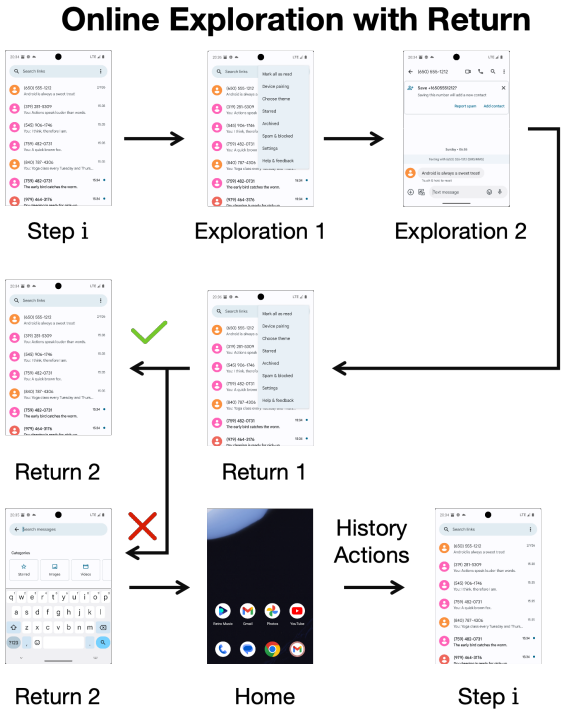


Figure 6: Efficient rollback for online exploration. Exploratory interactions may lead to different UI branches; some can be reverted via back navigation, while others introduce irreversible context changes. MobileExplorer restores the exploration starting UI state through trace-guided rollback and screen verification.

or state duplication. In contrast, MobileExplorer operates on a live interactive system, where each exploratory action directly modifies the real UI state. To preserve consistency between the reasoning input screen and the execution context, the agent must return to the exploration starting UI state after probing alternative UI branches.

5.3.1 Challenge. Rollback in mobile GUI exploration is fundamentally difficult due to the partially observable and non-deterministic nature of UI transitions. Unlike simulated environments with explicit state cloning, a real mobile interface exposes only the rendered screen, while the underlying system state remains hidden. As a result, the interaction process is better modeled as a partially observable Markov decision process (POMDP) rather than a fully observable and reversible Markov chain.

In practice, UI navigation is not strictly reversible. The back operation may skip intermediate states, modal dialogs and pop-ups can alter the transition path, and asynchronous content loading may lead to state-dependent screen variations even under identical actions. Moreover, certain interactions (e.g., opening new pages or triggering system UI) introduce irreversible context changes that cannot be perfectly undone by a single navigation step. Consequently, naive backtracking cannot reliably restore the exact exploration starting UI state, requiring a rollback mechanism that is both efficient and robust to non-deterministic transitions.

5.3.2 Two-Level Rollback Strategy. As shown in Fig. 6, MobileExplorer adopts a two-level rollback strategy.

Level-1: Depth-bounded backtracking. We perform depth-bounded exploration with a fixed depth d. After finishing one exploratory branch, we issue back exactly d times to return to the starting page. We then verify rollback correctness by comparing the current screen with the exploration starting screen using perceptual hash (pHash) [29], a lightweight image similarity metric robust to minor visual variations (e.g., scrolling or dynamic content). If the pHash distance is below a threshold δ, the rollback is considered successful.

Level-2: Home-and-replay recovery. If Level-1 rollback fails (e.g., due to non-reversible transitions or unexpected UI states), we fall back to a robust recovery routine. The agent returns to the Home screen and replays the original reasoning actions that led to the exploration starting UI state. This recovery guarantees that exploration does not permanently drift the agent away from the main decision trajectory.

This two-level design keeps rollback fast in the common case (Level-1) while maintaining correctness under non-deterministic UI behaviors (Level-2), thereby preserving the latency-hiding benefit of parallel exploration.

5.4 Exploration Memory and Reasoning Augmentation

Exploration is performed at step i starting from the screen s_i, while the actual reasoning action leads the agent to a new screen s_{i+1}. As a result, not all exploration trajectories are directly relevant to the subsequent decision. A key design challenge is to align exploration knowledge with the actual reasoning progress, rather than blindly injecting all explored information into the prompt.

Algorithm 1 Exploration with Rollback

Require: Task $\mathcal{T}$, start screen s_i, clickable set $\mathcal{A}_i$, depth d, budget τ_i, threshold δ
Ensure: Exploration hints C_i and restored starting UI state

$C_i \leftarrow \emptyset$
Rank candidates $\mathcal{A}_i^{\text{cand}}$ by semantic similarity to $\mathcal{T}$
for all $a \in \mathcal{A}_i^{\text{cand}}$ **within** τ_i **do**
  Interact with a and explore transitions up to depth d
  Record discovered screens/elements into C_i
  Issue back for d steps (Level-1 rollback)
  Capture current screen s'
  if $\textsc{pHashDist}(s', s_i) > \delta$ **then**
    Go home and replay reasoning actions to recover s_i (Level-2 recovery)
  end if
end for
return C_i

Step Alignment via Relevance Matching. After executing the reasoning action and reaching s_{i+1}, MobileExplorer performs a relevance matching process between the current screen and previously explored observations. Specifically, the system compares the semantic similarity between UI elements observed during exploration and the elements visible on s_{i+1}. Only exploration results that are semantically aligned with the current UI context are retained, while irrelevant branches are discarded. This prevents outdated or off-trajectory exploration knowledge from polluting the reasoning context.

Compact Exploration Memory. Instead of storing full interaction trajectories, which are noisy and redundant, MobileExplorer maintains a lightweight exploration memory that records visited UI elements, discovered screens, and their task relevance. This structured memory abstracts raw exploration into a compact representation suitable for on-device VLM prompting.

During exploration, task-related elements discovered along different branches are marked (e.g., highlighted regions on screenshots) and associated with textual descriptions such as newly revealed pages, actionable controls, and observed interaction outcomes. This marking mechanism preserves spatial grounding while keeping the memory size small.

Hint Generation and Context Augmentation. Based on the aligned exploration memory, the system summarizes key observations into concise textual hints and optionally masked or highlighted visual cues. These hints are appended to the prompt of the next reasoning step as exploration-derived context C_i.

Consequently, the VLM no longer reasons solely over the static screenshot s_{i+1}, but over an augmented context that integrates both the current UI and the aligned local interaction structure discovered during exploration. This enriched context reduces uncertainty about hidden UI branches and mitigates repeated trial-and-error interactions.

By combining step-aligned filtering, compact memory abstraction, and structured hint augmentation, MobileExplorer converts raw exploration trajectories into task-relevant knowledge, improving reasoning accuracy while maintaining low on-device overhead for long-horizon mobile GUI tasks.

6 EVALUATION

6.1 Experiment Setup

Benchmark. We evaluate MobileExplorer on AndroidWorld [14], a task-oriented mobile GUI interaction benchmark that requires multi-step reasoning over real smartphone interfaces. Unlike static GUI understanding datasets, AndroidWorld emphasizes closed-loop interaction, where the agent must continuously perceive the screen, reason about task progress, and issue sequential actions to reach the goal state. This setting better reflects practical mobile assistant scenarios, where interaction efficiency and system responsiveness are critical.

Devices. We deploy MobileExplorer on multiple real on-device platforms, including Samsung Galaxy S24 and Google Pixel 9 smartphones (12 GB RAM, 256 GB storage), an NVIDIA Jetson Xavier NX (16 GB) representing an edge AI device, and a MacBook Air M4 (24 GB) as a portable on-device computing platform.

Implementation. MobileExplorer is implemented as an end-to-end mobile GUI agent system. During evaluation, AndroidWorld tasks are executed on real smartphones while preserving natural interaction timing. To obtain stable latency measurements without interference from resource contention between GUI rendering and model inference, we adopt a dual-device execution setup in which one device runs the interaction tasks and provides GUI states, while the other performs perception and decision inference. We empirically verify that the measured step latency and end-to-end latency closely match those observed when running on a single device in isolation, indicating that the deployment strategy does not distort system timing. All perception and reasoning modules are executed on mobile hardware, ensuring that the evaluation captures the complete perception–reasoning–action loop under real device constraints.

Baselines. We compare MobileExplorer with the following representative baselines.

- **LLM-based agent**: A text-centric mobile GUI agent that relies on accessibility trees and performs step-wise reasoning using an LLM.
- **VLM-based agent**: A screenshot-based agent that uses a VLM to reason over the current screen in a sequential perception–reasoning–action loop.
- **Input-pruning VLM agent**: Agent that reduces visual token overhead via screenshot or token pruning [13].
- **Offline exploration agent**: Agent that construct knowledge through offline exploration [23, 30, 31].

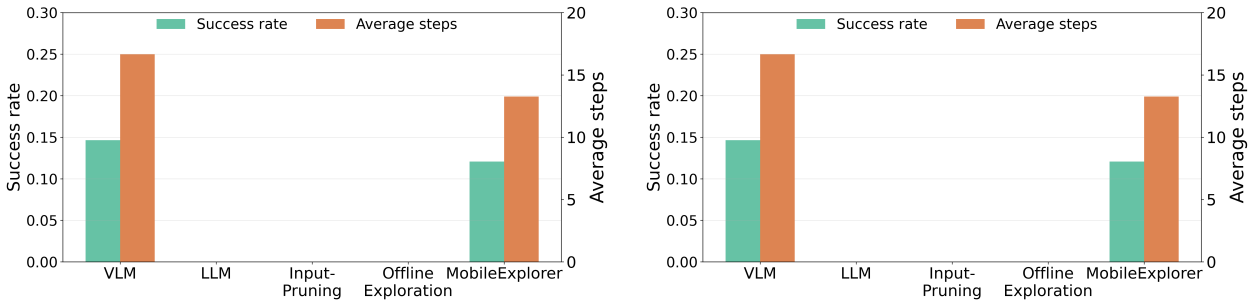
Evaluation metrics. We evaluate MobileExplorer in terms of both effectiveness and system efficiency on real on-device execution. **Success rate** measures the percentage of tasks successfully completed within a predefined step budget, indicating the overall task-solving capability of the agent. **Total steps** denotes the number of interaction steps required to finish a task, reflecting decision efficiency and the amount of trial-and-error during long-horizon execution. **Latency** is evaluated at two granularities. Step latency is defined as the time of a single interaction cycle, measured from capturing the current screenshot s_i to completing the execution of the selected action a_i. This includes perception, model reasoning, online exploration, and action execution. End-to-end latency is defined as the total task completion time from task initialization to the final successful state, i.e., the sum of all step latencies across the task.

6.2 Overall Performance

We evaluate MobileExplorer on the Android World benchmark under a fully on-device setting. Compared with baseline agents, MobileExplorer consistently improves task success while reducing interaction cost.

Overall, MobileExplorer achieves a success rate of X%, outperforming the VLM-based baseline by X% and the LLM-based agent by X%. More importantly, our method significantly reduces the average number of interaction steps from X to X, indicating that exploration-derived hints help the agent avoid unnecessary trial-and-error operations in long-horizon tasks.

In terms of efficiency, MobileExplorer lowers the average step latency to X s and reduces the total task completion time (end-to-end latency) by X% compared to the standard step-wise VLM agent. This improvement comes from overlapping online exploration with model reasoning, which converts otherwise idle reasoning time



(a) Success rate and average steps. **(b) LRW dataset**

Figure 7: Overall performance on AndroidWorld.

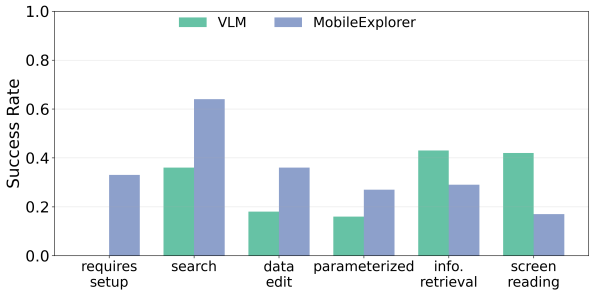


Figure 8: Success rate comparison across task categories.

into useful environment interaction without introducing extra user-perceived delay.

These results demonstrate that latency-bounded online exploration not only enhances decision quality but also improves system efficiency for multi-step mobile GUI tasks on resource-constrained devices.

6.3 Understanding Performance of MobileExplorer

Figure 8 breaks down performance by task category. MobileExplorer significantly improves navigation- and interaction-intensive tasks, including *requires_setup*, *search*, *data_edit*, and *parameterized*. These tasks require discovering hidden UI elements or exploring multiple branches, where time-budgeted exploration provides additional structural cues.

In contrast, performance decreases on perception-driven tasks such as *information_retrieval* and *screen_reading*, which primarily depend on direct visual interpretation rather than UI exploration.

Overall, the results indicate that online exploration is most beneficial for tasks involving structural UI uncertainty.

6.4 Memory and Energy Overhead

We evaluate the additional memory footprint and energy consumption introduced by the online exploration module, including the exploration controller, rollback mechanism, and exploration memory buffer.

As shown in Figure **??**, the memory overhead of all components remains modest, with the peak additional memory usage below X MB. In particular, the exploration controller consumes X MB, the rollback buffer requires X MB for storing screenshots and traces, and the compact exploration memory adds only X MB. This overhead is small compared to the memory consumption of on-device VLM inference and does not affect the feasibility of deployment on commodity mobile devices.

We further measure the runtime energy consumption of our system with and without the exploration module, as illustrated in Figure **??**. The proposed design introduces only a marginal increase in energy usage, with an average additional power consumption of X W during inference. This is because exploration operations consist of lightweight UI interactions and embedding computations, which are significantly less energy-intensive than VLM reasoning. Overall, the slight energy overhead is well justified by the reduction in total steps and end-to-end latency achieved by MobileExplorer.

6.5 Case Study

7 DISCUSSION

Discuss the limitation and future work.

8 CONCLUSION

In this paper, we introduce MobileExplorer, a new xx

REFERENCES

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025).

[3] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. 2024. Seeclick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9313–9332.

[4] Gaole Dai, Shiqi Jiang, Ting Cao, Yuanchun Li, Yuqing Yang, Rui Tan, Mo Li, and Lili Qiu. 2025. Advancing mobile gui agents: A verifier-driven approach to practical deployment. *arXiv preprint arXiv:2503.15937* (2025).

[5] Tinghe Ding. 2024. Mobileagent: enhancing mobile control via human-machine interaction and sop integration. *arXiv preprint arXiv:2401.04124* (2024).

[6] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[7] Google Android Developers. [n. d.]. Android Debug Bridge (adb). https://developer.android.com/tools/adb. ([n. d.]).

[8] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14281–14290.

[9] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. 2024. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1119–1133.

[10] Gang Li and Yang Li. 2022. Spotlight: Mobile ui understanding using vision-language models with a focus. *arXiv preprint arXiv:2209.14927* (2022).

[11] Ning Li, Xiangmou Qu, Jiamu Zhou, Jun Wang, Muning Wen, Kounianhua Du, Xingyu Lou, Qiuying Peng, and Weinan Zhang. [n. d.]. MobileUse: A Hierarchical Reflection-Driven GUI Agent for Autonomous Mobile Operation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025b. URL https://openreview. net/forum*.

[12] Wei Li, William E Bishop, Alice Li, Christopher Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. 2024. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems* 37 (2024), 92130–92154.

[13] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19498–19508.

[14] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. 2024. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573* (2024).

[15] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. [n. d.]. Android in the wild: A large-scale dataset for android device control, 2023. *URL https://arxiv. org/abs/2307.10088* ([n. d.]).

[16] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).

[17] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[18] Shizuo Tian, Hao Wen, Yuxuan Chen, Jiacheng Liu, Shanhui Zhao, Guohong Liu, Ju Ren, Yunxin Liu, and Yuanchun Li. 2025. AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. *arXiv preprint arXiv:2512.10371* (2025).

[19] Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. 2025. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544* (2025).

[20] Junyang Wang, Haiyang Xu, Haitao Jia, Xi Zhang, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. 2024. Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration. *Advances in Neural Information Processing Systems* 37 (2024), 2686–2710.

[21] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 543–557.

[22] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, et al. 2025. Autodroid-v2: Boosting slm-based gui agents via code generation. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 223–235.

[23] Bin Xie, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Jie Liu, Min Zhang, and Liqiang Nie. 2025. Gui-explorer: Autonomous exploration and mining of transition-aware knowledge for gui agent. *arXiv preprint arXiv:2505.16827* (2025).

[24] Yifan Xu, Xiao Liu, Xinghan Liu, Jiaqi Fu, Hanchen Zhang, Bohao Jing, Shudan Zhang, Yuting Wang, Wenyi Zhao, and Yuxiao Dong. 2025. Mobilerl: Online agentic reinforcement learning for mobile gui agents. *arXiv preprint arXiv:2509.18119* (2025).

[25] Yifan Xu, Xiao Liu, Xueqiao Sun, Siyi Cheng, Hao Yu, Hanyu Lai, Shudan Zhang, Dan Zhang, Jie Tang, and Yuxiao Dong. 2025. Androidlab: Training and systematic benchmarking of android autonomous agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2144–2166.

[26] Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, et al. 2025. Step-gui technical report. *arXiv preprint arXiv:2512.15431* (2025).

[27] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. 2025. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144* (2025).

[28] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-ui: Grounded mobile ui understanding with multimodal

llms. In *European Conference on Computer Vision*. Springer, 240–255.

[29] Christoph Zauner. 2010. Implementation and Benchmarking of Perceptual Image Hash Functions. (2010).

[30] Yao Zhang, Zijian Ma, Yunpu Ma, Zhen Han, Yu Wu, and Volker Tresp. 2025. Webpilot: A versatile and autonomous multi-agent system for web task execution with strategic exploration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 23378–23386.

[31] Shanhui Zhao, Hao Wen, Wenjie Du, Cheng Liang, Yunxin Liu, Xiaozhou Ye, Ye Ouyang, and Yuanchun Li. 2025. LLM-Explorer: Towards Efficient and Affordable LLM-based Exploration for Mobile Apps. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 589–603.

[32] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. 2025. MAI-UI Technical Report: Real-World Centric Foundation GUI Agents. *arXiv preprint arXiv:2512.22047* (2025).